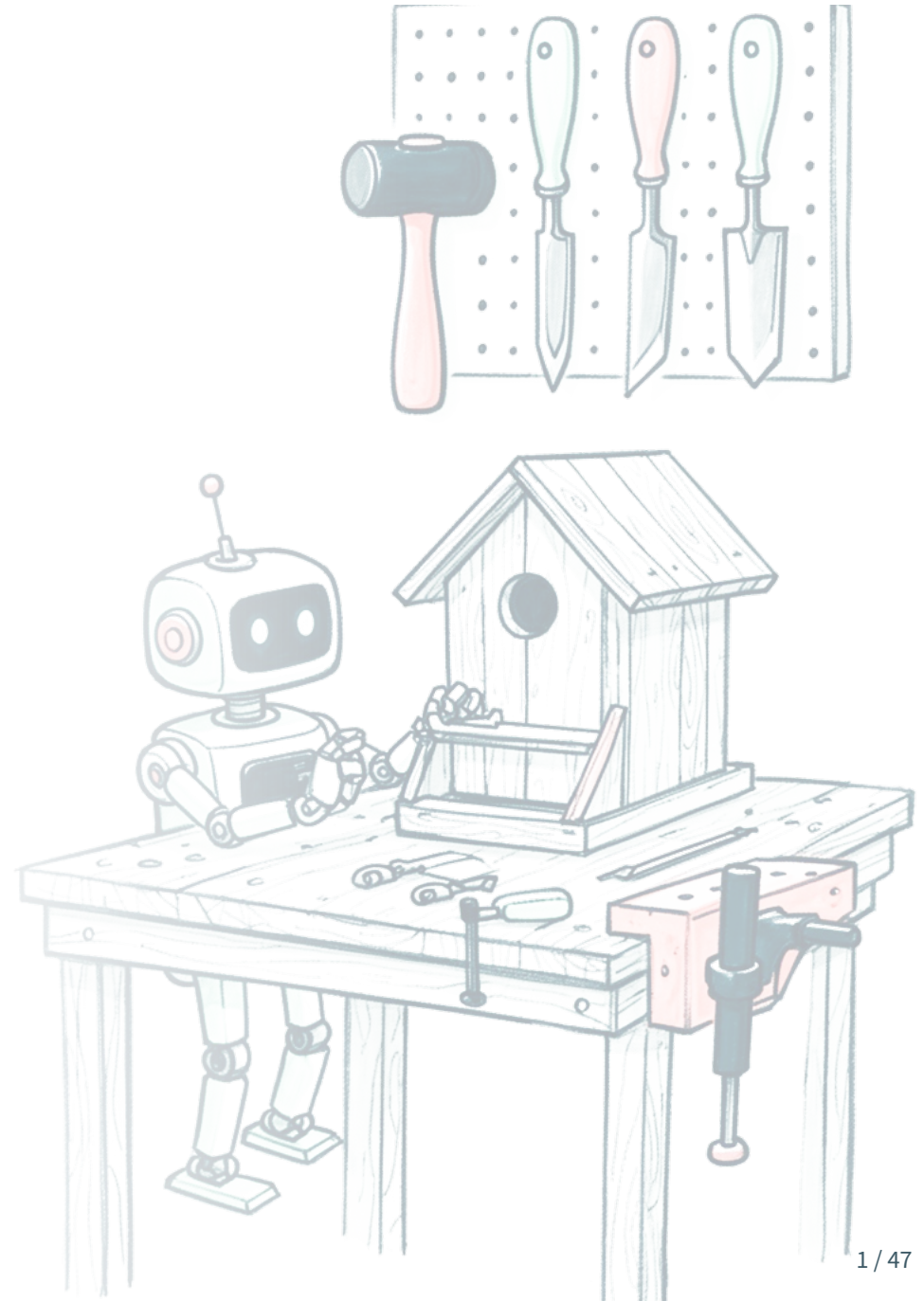


Agentic AI in der Praxis

Vom Spec zum produktiven Workflow, mit Übungen am eigenen Rechner

Illustrations AI-generated · Flux 2 (flux2-dev)



Zwei Beispiele aus unserer eigenen Arbeit

Christian Uhl · ZAD, Uni Gießen

- Unbekannter Chip ohne Firmware
- Ein einzelner Fehler hätte ihn zerstört
- Sorgfältig war zu langsam, schnell zu riskant
- Nach 24 Stunden: FIDO2-Firmware, spec- und testgetrieben

Tobias Weiß · DevOps, Uni Marburg

- openDesk Edu: Dienste für Forschung, Lehre, Verwaltung
- Wissens-Pipelines und IT-Betrieb agentisch
- Open-Source-Software mit Open-Source-Werkzeugen

Ihre Reihe: Blitzlicht in dreißig Sekunden pro Person

- Name, Fachrichtung, ein Thema für eine Literaturrecherche
- Selbsteinschätzung KI: eher Anfang oder eher fortgeschritten?
- Wir sammeln die Wünsche am Whiteboard
- Am Ende prüfen wir sie gegen die Outcomes

Der Vormittag liefert ein Werkzeug, keinen fertigen Bot

- Ein Research-Repo mit laufender Pipeline
- Einen Spec-Change, live umgesetzt
- Werkzeuge für Modelle und Harnesses statt Trendlisten
- Nicht dabei: der voll-autonome Assistent

Drei Stunden, zwei Hands-Ons, dazwischen die Theorie

Zeit	Block	Wer
09:00	Ankommen, Vorstellung, Blitzlicht	Christian, Tobias
09:15	Grundlagen	Christian
09:30	Foundation Models	Christian, Tobias
09:50	Open-Source Toolbox	Tobias, Christian
10:20	Hands-On: Erste Schritte	Tobias
10:45	Spezifikation und Token	Christian, Tobias
11:00	Hands-On: Mit Spec	Christian
11:30	Outcomes, Q&A, Wrap Up	Tobias, Christian

1

Grundlagen: worüber wir sprechen

Bei diesen fünf Begriffen legen wir uns für heute fest

Begriff	Worüber gestritten wird	Unsere Festlegung
Agent	von Chatbot bis autonomes System	Das Modell bestimmt die nächsten Schritte
Reasoning	ob das Denken ist	mehr Rechenzeit zur Laufzeit
Halluzination	Begriff umstritten, eher Konfabulation	unbelegte Ausgabe, Grund für die Tests
Open Source	offene Gewichte sind nicht OSI	wir meinen offene Gewichte
Reproduzierbarkeit	gleiche Eingabe, anderes Ergebnis	geprüft wird das Ergebnis, nicht der Weg

Ein agentisches System besteht aus vier Bausteinen

Baustein	Was es ist	Was es nicht ist
Modell	sagt das nächste Token voraus	kein Wissensspeicher, keine Datenbank
Kontext	was im Eingabefenster steht	kein Gedächtnis, endet mit der Sitzung
Werkzeuge	Dateien, Shell, Netz, Schnittstellen	keine Rechte jenseits des Freigegebenen
Harness	stellt Kontext, ruft Werkzeuge, speist zurück	trifft die Entscheidungen nicht selbst

2

Spezifikation, Vertrag, Tests

Fragen oder Beauftragen

- Fragen: „Fassen Sie diese Papers zusammen.“
- Beauftragen: „Extrahieren Sie diese fünf Felder aus jedem Paper.“



Die Spezifikation regiert, der Test entscheidet



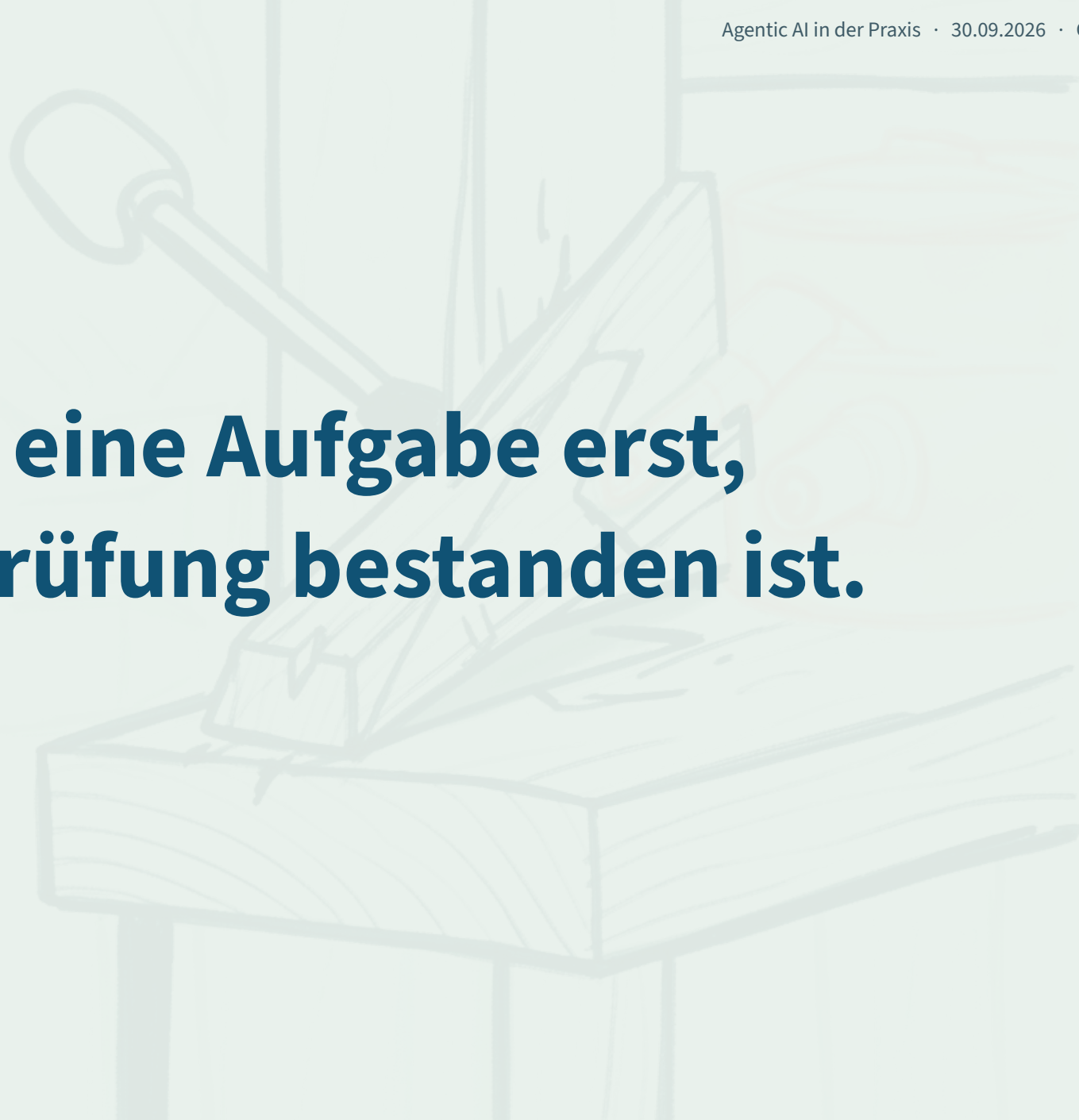
- Jede Ebene ist eine Datei, kein Gespräch
- Der Vertrag ist eine Spezifikation für eine Änderung
- Der Vertrag nennt das Kriterium, der Test wendet es an
- Was die Tests zeigen, ändert die Spezifikation

Jede der drei Ebenen legt etwas anderes fest

Ebene	Legt fest	Im Repository
Spezifikation	was gelten soll, prüfbar formuliert	specs/, AGENTS.md
Vertrag	was sich jetzt ändert, und woran man es misst	ein Change mit Vorschlag und Aufgaben
Tests	ob die Anforderung erfüllt ist	Testsuite, Validierung, CI



**Erledigt ist eine Aufgabe erst,
wenn die Prüfung bestanden ist.**



3

Foundation Models: aktuelle Entwicklungen

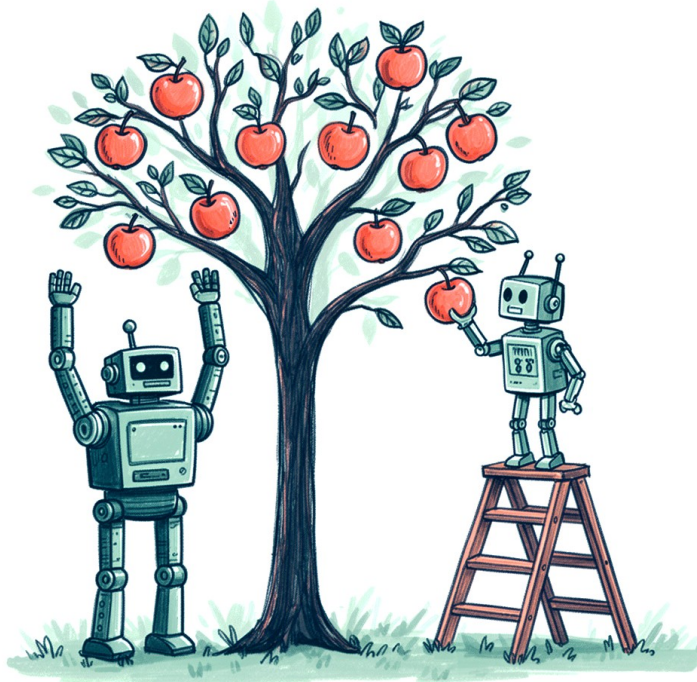
Sechs Entwicklungen prägen die Modelle im Jahr 2026

- Reasoning ist reif: planen statt vervollständigen
- Kontext wächst von 200.000 auf über eine Million Token
- MCP wird der Standard für Werkzeugzugriff
- Werkzeugaufrufe sind produktionsreif
- Offene Gewichte holen auf, die Kosten fallen
- Lokal und souverän betreiben ist eine echte Option

Das passende Modell hängt von der Aufgabe ab

Aufgabe	Derzeit stark	Anmerkung
Freie Texte, Übersetzung	GPT-6	derzeit führend bei Prosa
Räumliche Aufgaben, 3D	GPT-6	in eigenen Tests überzeugend
Code, größere Umbauten	Claude, zunehmend GLM	Claude teuer, oft auf Anhieb erfolgreich
Code gezielt umschreiben	Qwen3.8-Flash-Next, GLM	offene Gewichte, schnell, günstig
Abläufe mit Werkzeugen	GLM 4.7, Claude	verlässliche Werkzeugaufrufe
Sensible Forschungsdaten	lokal: Ollama, vLLM	die Daten verlassen das Haus nicht
Viele gleichartige Fälle	DeepSeek V4 Flash	Geschwindigkeit und Kosten entscheiden

Das Harness bestimmt, was das Modell erreichen kann



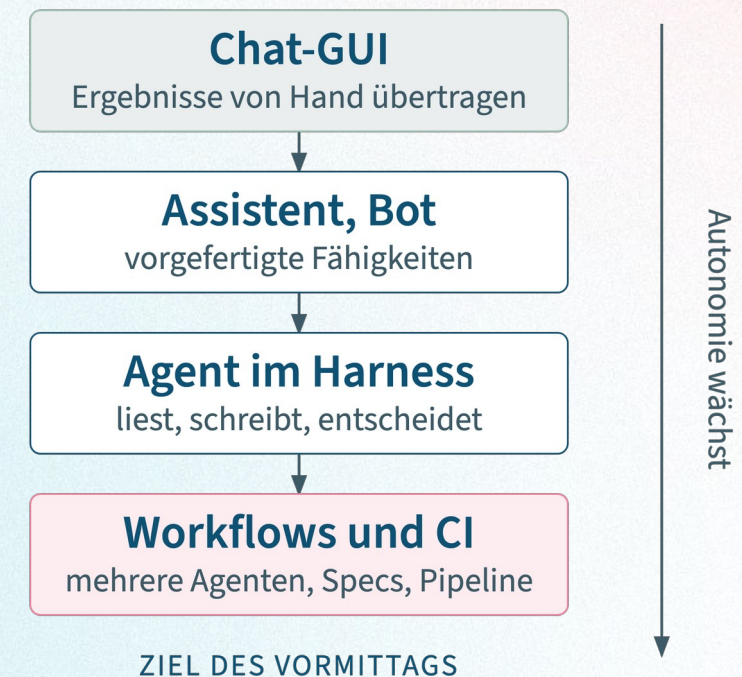
- Einzelne Frage: das Modell prägt die Antwort allein
- Mehrere Schritte: das Harness setzt die Reichweite
- Jeden Schritt darin wählt weiterhin das Modell
- Günstiges Modell im guten Harness vor teurem im schlechten

4

Open-Source Toolbox: Harnesses als Steuerungsebene

Agentisch ist nicht das Terminal, sondern die Autonomie

- Auch eine GUI kann agentisch sein
- Das Terminal ist heute der reifste Ort dafür
- Prüfbar, wiederholbar, automatisierbar



Zwei offene Harnesses, der Einsatz entscheidet

OpenCode, der Allrounder

- Agent im Terminal, Modell je Agent
- LSP, Plugins, Skills, MCP
- oh-my-opencode: Routing, AST-Grep, Background-Agents
- Für tägliche Coding-Agents

pi, der Minimalist

- Skills, Prompt-Templates, Packages
- „Adapt pi, nicht umgekehrt“
- Für kontrollierte, minimalistische Workflows

pi bleibt klein, alles andere kommt als Package

Kategorie	Beispiele
Skills	superpowers, CVE-Scanner, AGENTS.md-Audit
Agenten	subagents, until-done, plan-mode, evaluate
Stil, Werkzeuge, Suche	caveman, ponytail, mcp-Adapter, websearch
UI	webui, desktop-ui, glimpseui, chrome
Absicherung	tool-repair, retry, permissions, secrets, rtk

Pause.
Zehn Minuten.



5

Hands-On: Erste Schritte

Erst planen, dann bauen

- Plan-Mode: der Agent liest, fragt und schlägt vor
- Nichts wird geändert, bis Sie zustimmen
- Build-Mode: der Agent setzt den freigegebenen Plan um
- Den Plan lesen ist der schnellste Weg zum Verständnis

Übung 1: Ihr Thema X, direkt an den Agenten, fünfzehn Minuten

Der Agent übernimmt	Heißt konkret
Entdecken	neue Veröffentlichungen zu Thema X aufspüren
Einordnen	Funde den Themenfeldern zuordnen
Prüfen	Quellen verifizieren, Erfindungen aussortieren
Berichten	Überblick, Trends, Kurz-Briefings



Sie entscheiden, was relevant ist.

Der Agent übernimmt die Fleißarbeit.

AGENTS.md: Regeln, die der Agent bei jedem Start liest

- Erzeugtes wird nie von Hand geändert
- Nichts wird erfunden, jede Quelle muss auflösbar sein
- Nach jeder Änderung die komplette Prüfung, dann Commit
- Geführt per Datei, nicht per Prompt

6

Spezifikation und Token

Im Projekt liegen die drei Ebenen in drei Ordnern

Ort	Was dort liegt	Rolle
openspec/specs/	das dauerhaft gültige Verhalten	Spezifikation
openspec/changes/	die eine anstehende Änderung	Vertrag, genau das bekommt der Agent
openspec/archive/	alle abgeschlossenen Änderungen	Begründung, warum so entschieden

Spec-Kit baut Neues, OpenSpec ändert Bestehendes

	Spec-Kit	OpenSpec
Grundidee	was gebaut werden soll	wie sich Bestehendes ändert
Typischer Fall	Neubau, neue Funktionen	laufende Weiterentwicklung
Quelle der Wahrheit	eine Spezifikation je Vorhaben	dauerhafte Spezifikation plus Änderungen
Ablauf	specify, clarify, plan, tasks	propose, apply, sync, archive
Aufwand	höher, mehr Prozessführung	niedriger

Ein fertiger Change steht als Vorlage bereit

- Repo ai-literacy-research: über 9.600 Papers, 20 Kategorien
- openspec/specs/: Corpus, Discovery, Analyse
- Change add-research-gap-analysis: proposal, design, specs, tasks
- Gap-Score: $0,6 \times \text{Dünne} + 0,4 \times \text{Momentum} + \text{Floor-Bonus}$

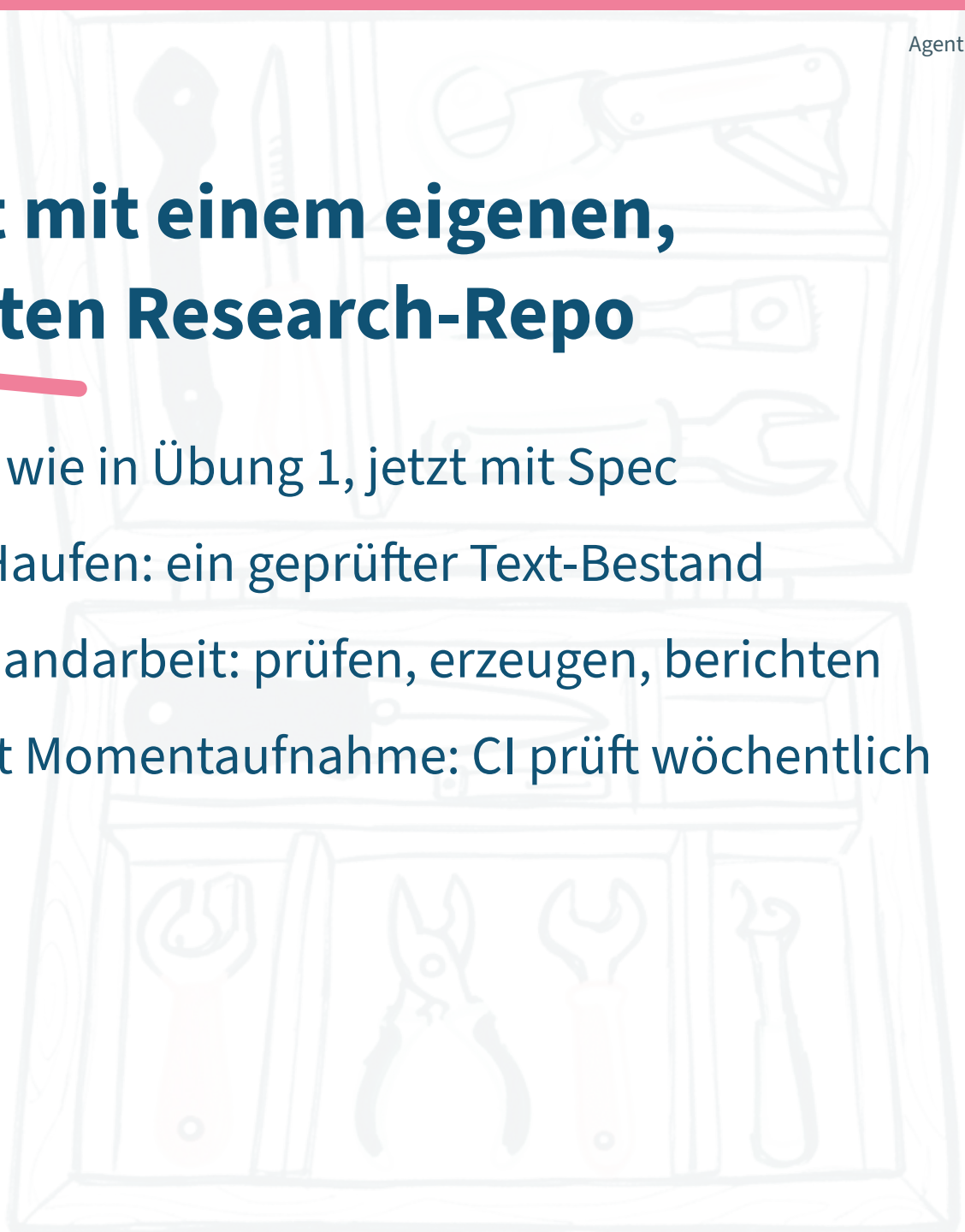
Drei- bis fünfmal weniger Token, bei gleicher Qualität

- Routing: das richtige Modell pro Aufgabe
- Caching und Kontext-Hygiene: nur behalten, was zählt
- Klein ist schön: kleines Modell, kleine Aufgabe
- Skills und Specs schreiben statt jedes Mal erklären
- Wissen speichern, nicht Prompt-Stücke

7

Hands-On: Mit Spec

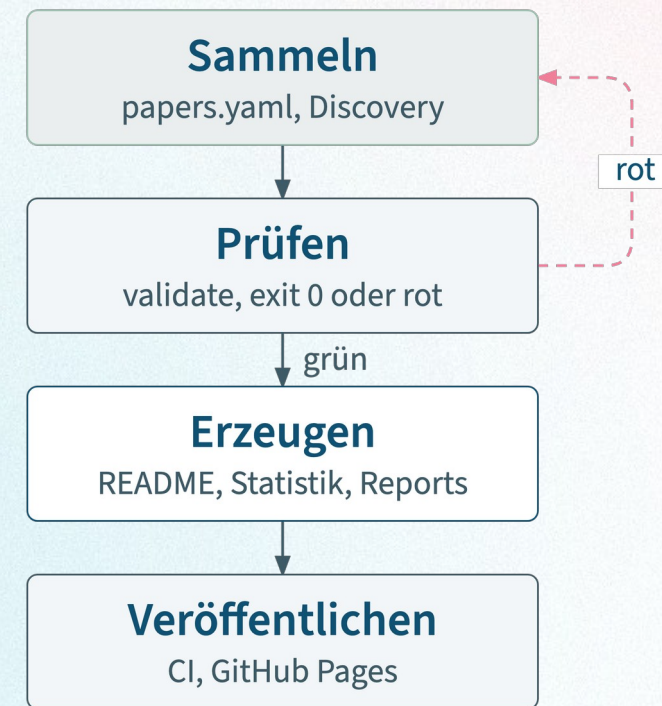
Jeder geht mit einem eigenen, CI-validierten Research-Repo



- Gleiche Frage wie in Übung 1, jetzt mit Spec
- Corpus statt Haufen: ein geprüfter Text-Bestand
- Regeln statt Handarbeit: prüfen, erzeugen, berichten
- Lebendig statt Momentaufnahme: CI prüft wöchentlich

In der Pipeline hat jede Stufe genau eine Aufgabe

- Ein Werkzeug, ein Job
- Text ist die Schnittstelle, für Mensch und Agent
- Die Pipeline ist der Richter, nicht die Meinung



Der Jump-Start dauert fünf Schritte

- Kopie holen: das Demo-Repo wird Ihres
- Themenfeld setzen: Ihre Ordnung für Thema X
- Seeden: drei bis fünf Treffer aus Übung 1
- Laufen lassen: prüfen, erzeugen, berichten
- Veröffentlichen: das Ergebnis geht online

Übung 2: eine überprüfbare Anforderung für Ihr Thema

- Klein halten: eine Auswertung, eine Kennzahl
- Fertig heißt: die Prüfung läuft durch
- `openspec new change thema-x`
- `openspec validate --changes`
- Ohne OpenSpec: dieselbe Struktur als Markdown

Schon das Mindestziel zeigt, dass die Methode sitzt

Ebene	Mindestziel	Vollständig
Spezifikation	ein Satz, der widerlegt werden kann	Szenario: Ausgangslage, Eingriff, Ergebnis
Vertrag	eine Aufgabenliste mit drei Punkten	Vorschlag, Entwurf, Spezifikation, Aufgaben
Tests	eine Prüfung, die fehlschlagen könnte	läuft automatisch und besteht

Pause.

Fünf Minuten.



8

Outcomes: Ihre Ergebnisse

Gezeigt wird ein eigenes Repo mit eigenem Change

- Drei bis vier Freiwillige, je drei bis vier Minuten
- Pipeline grün, Change validiert, live oder als Screenshot
- Bewertet an der Pyramide: Spec, Change, grüner Check
- Das Publikum reviewt mit

9

Q&A, Diskussion, Wrap Up

So übertragen Sie das Muster auf eigene Projekte

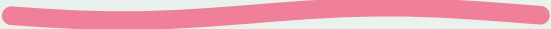
- Klein starten: eine Spec für ein vorhandenes Modul
- Erster Change mit drei Tasks, den Agent umsetzen lassen
- Tests und CI als Pflicht, iterieren bis grün
- Wissen persistieren, ab der ersten Sitzung
- Erkenntnisse aus dem Betrieb werden neue Changes

Die Werkzeuge des Vormittags zum Nachinstallieren

Werkzeug	Adresse
OpenCode	github.com/sst/opencode
pi	pi.dev
zot	zot.sh
OpenSpec	npmjs.com/package/openspec
SAIA-Plugin für pi	codeberg.org/tobias-weiss-ai-xr/pi-saia-plugin
SAIA-Plugin für OpenCode	github.com/tobias-weiss-ai-xr/opencode-saia-plugin
oh-my-opencode	github.com/code-yeongyu/oh-my-opencode

Repositories und Skills zum Weiterarbeiten

Name	Adresse
skeleton-research	github.com/tobias-weiss-ai-xr/skeleton-research
ai-literacy-research	github.com/tobias-weiss-ai-xr/ai-literacy-research
Superpowers	github.com/obra/superpowers
rtk	github.com/rtk-ai/rtk
ponytail	github.com/DietrichGebert/ponytail
caveman	github.com/JuliusBrussee/caveman



Vielen Dank!
Fragen gern als GitHub-Issue.

CC BY 4.0 · Christian Uhl · Tobias Weiß

Quellen und Links

- [1] Eigene Erfassung, Stand 09/2026
- [2] Eigene Beobachtung, kein kontrollierter Vergleich
- [3] Eigene Erfassung
- [4] 26 Packages, eigene Installation, Stand 09/2026 — <https://pi.dev/packages>
- [5] GitHub Spec Kit — <https://github.com/github/spec-kit>
- [6] Fission AI OpenSpec — <https://github.com/Fission-AI/OpenSpec>
- [7] Eigene Messung, kein kontrollierter Vergleich
- [8] OpenCode — <https://github.com/sst/opencode>
- [9] pi — <https://pi.dev>
- [10] zot — <https://zot.sh>
- [11] OpenSpec — <https://www.npmjs.com/package/openspec>
- [12] pi-saia-plugin — <https://codeberg.org/tobias-weiss-ai-xr/pi-saia-plugin>
- [13] opencode-saia-plugin — <https://github.com/tobias-weiss-ai-xr/opencode-saia-plugin>
- [14] oh-my-opencode — <https://github.com/code-yeongyu/oh-my-opencode>
- [15] skeleton-research — <https://github.com/tobias-weiss-ai-xr/skeleton-research>
- [16] ai-literacy-research — <https://github.com/tobias-weiss-ai-xr/ai-literacy-research>
- [17] Superpowers — <https://github.com/obra/superpowers>
- [18] rtk — <https://github.com/rtk-ai/rtk>
- [19] ponytail — <https://github.com/DietrichGebert/ponytail>
- [20] caveman — <https://github.com/JuliusBrussee/caveman>